Kanban e Gestão de Fluxo

Processos de Software — Semana 4

Prof. Fernando · Engenharia de Software · UFRN · 2026.1

Do Toyota Production System ao quadro Kanban em software

Recapitulando a aula anterior

XP e **Lean** convergem em um ponto: **ciclos curtos de feedback** e **eliminação de desperdício**.

- XP nos deu as *práticas habilitadoras* (testes, refatoração, CI) que tornam mudanças baratas
- Lean nos deu os *princípios* (eliminar muda, ver o todo, decidir tarde, entregar rápido)

Mas uma pergunta ficou no ar:

> *Como é que, na prática, a gente visualiza e gerencia o fluxo de valor de um time de software?*

A resposta começou na Toyota dos anos 1950.

Kanban: origem na Toyota

- *kanban* em japonês = **cartão, sinal visual**
- Criado por **Taiichi Ohno** nos anos 1950 como parte do TPS
- Inspiração: supermercados americanos
  - Quando a prateleira esvaziava, um cartão sinalizava ao depósito para repor
  - **Nada era produzido sem um sinal de demanda** (pull)
- Contraponto à produção em massa do Fordismo (push)

> Ohno: *"Observei atentamente como o gerente de supermercado operava a loja. Ele não pedia ao fornecedor para reabastecer até que o produto começasse a acabar na prateleira."*

Lean Primer, Larman & Vodde 2009, p. 31-33

Kanban em software

- David J. Anderson (2010) adaptou os princípios do TPS para desenvolvimento de software
- O livro seminal: **"Kanban: Successful Evolutionary Change for Your Technology Business"**
- Filosofia: **comece onde você está** — não exige reestruturação da equipe
- Focado em **melhoria evolutiva**, não revolução

Kanban em software NÃO É:

- Um framework como Scrum (não tem papéis, cerimônias, iterações obrigatórias)
- Um método prescritivo ("você deve fazer X, Y, Z")
- Uma ferramenta (Trello, Jira, GitHub Projects são ferramentas que implementam Kanban)
- Apenas um quadro com post-its

Kanban é um **método de gerenciamento de fluxo** que pode ser aplicado a qualquer processo.

Os 4 princípios fundamentais

Os princípios do Kanban orientam **como começar** a aplicar o método:

1. **Comece com o que você faz agora**
   - Entenda seu processo atual, não o descarte
2. **Concorde em buscar mudança evolutiva e incremental**
   - Pequenas melhorias contínuas (← conexão direta com kaizen)
3. **Respeite o processo, papéis e responsabilidades atuais**
   - Sem reorganizações traumáticas
4. **Encoraje atos de liderança em todos os níveis**
   - Qualquer pessoa pode propor melhorias

> É a abordagem mais "gentil" entre os métodos ágeis — não exige transformação radical.

Anderson, 2010

As 6 práticas do Kanban

As práticas são **como fazer** Kanban no dia-a-dia:

1. **Visualizar o fluxo de trabalho**
2. **Limitar o trabalho em progresso (WIP)**
3. **Gerenciar o fluxo**
4. **Tornar as políticas explícitas**
5. **Implementar loops de feedback**
6. **Melhorar colaborativamente, evoluir experimentalmente**

Vamos ver as três principais em detalhe.

Prática 1: Visualizar o fluxo

O trabalho invisível é o pior tipo de trabalho:

- Não pode ser priorizado
- Não pode ser medido
- Gera estresse oculto
- Esconde gargalos

O quadro Kanban básico:

To Do	Doing	Review	Done
tarefa A	tarefa C	tarefa B	tarefa D
tarefa E			

> *"Tornar o trabalho visível" significa que qualquer pessoa passando pelo quadro entende o estado do sistema em 30 segundos.*

Princípios visuais (do Lean):

- **Grande** (visível de longe)
- **Físico** (cards que você pode tocar — ou ao menos bem visíveis digitalmente)
- **Colorido** (classes de serviço, urgência)

Lean Primer, p. 32-34

Prática 2: Limitar WIP (Work In Progress)

A regra mais **controversa** e **poderosa** do Kanban:

> **Cada coluna tem um limite máximo de cartões.**

Por que limitar WIP?

Sem limite: o time puxa novas tarefas sempre que possível → muitas coisas "começadas", poucas "terminadas".

Com limite: quando uma coluna está cheia, ninguém pode puxar mais nada → força o time a **terminar antes de começar**.

Exemplo:

To Do	Doing (WIP=2)	Review (WIP=1)	Done
A, B, E	C, D	F	G, H

Se alguém termina `C`, e quer começar `A`, **não pode** — `F` está bloqueando a Review. Solução: **ajudar** `F` a ser revisado. Isso elimina gargalos.

Por que WIP limits funcionam?

Lei de Little (teoria de filas)

$$\text{Lead Time} = \frac{\text{WIP}}{\text{Throughput}}$$

- **Lead time**: tempo do pedido à entrega
- **WIP**: trabalho em progresso
- **Throughput**: itens entregues por unidade de tempo

Conclusão: **se aumentar WIP sem aumentar throughput, lead time aumenta**.

O efeito prático:

- Menos WIP → foco, conclusão, feedback rápido
- Mais WIP → multitarefa, contexto perdido, nada fica pronto

> É a aplicação direta de "amplificar aprendizado" e "entregar rápido" dos Poppendieck.

Prática 3: Gerenciar o fluxo

Métricas fundamentais de fluxo:

Lead time: do momento em que o cliente pede até o momento em que entregamos

- *Do cliente, cliente que está esperando*

Cycle time: do momento em que começamos a trabalhar até o momento em que entregamos

- *Do time — o que o time pode otimizar*

Throughput: quantos itens entregamos por semana

Diferença entre lead time e cycle time = **tempo de espera em fila**

- Este é o desperdício principal a atacar!

Exemplo:

Se uma tarefa fica 10 dias no `To Do` e 4 dias em `Doing`:

- Lead time = 14 dias, Cycle time = 4 dias
- **Tempo em fila = 10 dias** (71% do lead time!)

O Cumulative Flow Diagram (CFD)

Gráfico de área empilhada mostrando quantidade de cartões em cada coluna ao longo do tempo.

O que observar:

- **Largura das faixas**: WIP por coluna
- **Faixa muito larga**: gargalo!
- **Distância vertical** entre curvas "Done" e "In progress" = lead time
- **Inclinação** da linha "Done" = throughput

```
WIP
 |
 |   ___Done_______
 |  /  ____________
 | /  / In Review
 |/  / ____________
 |  / / In Progress
 | / / ____________
 |/_/_/_To Do______
 |_______________
      Tempo →
```

> Ferramentas como GitHub Projects e Jira geram CFD automaticamente.

Classes de serviço

Nem toda tarefa tem a mesma urgência. Kanban define **classes de serviço**:

Classe	Característica	Política
Expedite	Emergência (produção quebrada)	Ignora WIP limits, passa na frente
Fixed date	Tem prazo legal/contratual	Agendada para garantir o prazo
Standard	Fluxo normal	FIFO, respeitando WIP
Intangible	Sem urgência (refatoração, docs)	Preenche capacidade ociosa

Por que explicitar?

Sem classes explícitas, toda tarefa parece "prioritária" → multitarefa → nada é entregue.

Com classes, o time sabe **quando** quebrar a regra e **quando** seguir o fluxo.

Scrum vs Kanban

Aspecto	Scrum	Kanban
Iterações	Sprints de duração fixa	Fluxo contínuo
Papéis	PO, SM, Dev Team	Nenhum prescrito
Cerimônias	Planning, Daily, Review, Retro	Nenhuma obrigatória
Mudanças no meio	Proibidas no sprint	Permitidas
Métrica principal	Velocity (pontos/sprint)	Lead time, throughput, WIP
Quadro	Reinicia a cada sprint	Contínuo
Abordagem	Revolucionária	Evolucionária

Scrumban (a abordagem do nosso curso):

- Scrum para **planejamento e estrutura** (sprints de 2 semanas, retrospectivas)
- Kanban para **gerenciar o fluxo de trabalho dentro do sprint**
- WIP limits + visualização explícita

Fluxo e o efeito "lago e rochas"

Lembram da metáfora do Lean Primer?

- **Lago** = tamanho do lote, tempo de ciclo, WIP
- **Rochas** = problemas e ineficiências ocultos
- Água alta (lotes grandes, WIP alto) → rochas invisíveis
- Água baixa (lotes pequenos, WIP baixo) → rochas aparecem

Em software:

Ao **limitar WIP agressivamente**, problemas que estavam escondidos ficam expostos:

- "Não temos ambiente de teste estável"
- "Nosso code review demora 3 dias"
- "O deploy é manual e ninguém sabe fazer direito"
- "Cinco pessoas sabem aquela parte do código, mas só uma realmente entende"

> A dor do WIP limit é o motor da melhoria contínua.

Lean Primer, p. 27-29

Ferramentas que implementam Kanban

Físicas

- Quadro branco com post-its ou cartões magnéticos (o ideal do Lean)
- Paredes com fita adesiva dividindo colunas

Digitais

- **GitHub Projects** (vamos usar no projeto do curso)
- Jira (mercado corporativo)
- Trello (simples, gratuito)
- Azure Boards
- Linear (moderno, minimalista)

Dica importante:

> A ferramenta **não é** o Kanban. Kanban é o **método**. Você pode ter Jira configurado como cascata disfarçada, e pode ter Kanban perfeito com post-its numa parede.

GitHub Projects na prática

Vamos usar no projeto do curso:

Configuração recomendada:

1. **Colunas**: `Backlog` → `To Do` → `In Progress` → `Review` → `Done`
2. **WIP limits**:
   - `In Progress`: 1 por pessoa do time (ex: 4 para time de 4)
   - `Review`: metade do time (ex: 2 para time de 4)
3. **Labels**:
   - `expedite`, `standard`, `intangible`
   - `feat`, `bug`, `chore`, `docs`
4. **Campos customizados**:
   - Estimativa (P/M/G ou Fibonacci)
   - Data de início (para calcular cycle time)

O quadro é do time. O time decide e evolui o processo.

DevOps: ponte natural do Kanban para a prática

Por que DevOps entra aqui?

Kanban otimiza o **fluxo**, mas o fluxo trava em lugares invisíveis:

- Deploy manual (gargalo enorme, difícil de ver)
- Testes manuais (não dá para aumentar throughput)
- Ambientes instáveis (defeitos mascarados)

DevOps = Kanban aplicado a operações + desenvolvimento

- **Integração contínua** → fluxo contínuo de código
- **Deploy automatizado** → elimina o gargalo de "entregar"
- **Infraestrutura como código** → ambientes reproduzíveis
- **Observabilidade** → feedback rápido sobre produção

> Próximas semanas: vamos aprofundar em DevOps, CI/CD, DORA metrics e práticas modernas.

Atividade prática (15 min)

Em grupo, usando o GitHub Projects do seu projeto:

1. **Criem** as colunas: Backlog, To Do, In Progress, Review, Done
2. **Adicionem** as 5 primeiras user stories do backlog da Sprint 1
3. **Definam WIP limits** para cada coluna (escrevam no nome da coluna)
4. **Criem labels** para classes de serviço
5. **Discutam**: qual é o "work in progress limit" que faz sentido para o seu time?

Entrega:

- Screenshot do quadro configurado
- Comentem no Discord #projeto

Recapitulando

Kanban em software:

- Método evolutivo de gerenciamento de fluxo
- 4 princípios: comece onde está, mudança incremental, respeite papéis, encoraje liderança
- 6 práticas: visualizar, limitar WIP, gerenciar fluxo, políticas explícitas, feedback, melhorar
- Métricas: lead time, cycle time, throughput, CFD

WIP limits são a regra mais poderosa:

- Forçam o time a terminar antes de começar
- Expõem gargalos e ineficiências (efeito "lago e rochas")
- Aplicação direta de Lei de Little

Scrumban: combinando o melhor de Scrum (estrutura de sprints) e Kanban (gestão de fluxo).

DevOps: natural evolução — Kanban aplicado também à entrega e operações.

Referências

- **Anderson, D. J.** (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.
- **Ohno, T.** (1988). *Toyota Production System: Beyond Large-Scale Production*. Productivity Press.
- **Larman, C. & Vodde, B.** (2009). *Lean Primer*. leanprimer.com (CC)
- **Reinertsen, D.** (2009). *The Principles of Product Development Flow*. Celeritas Publishing.
- **Poppendieck, M. & T.** (2003). *Lean Software Development*. Addison-Wesley.
- **Little, J. D. C.** (1961). *A Proof for the Queuing Formula: $L = \lambda W$*. Operations Research.